Protocol Execution – Soil Moisture Decline

Part 1: Protocol Execution & Data Acquisition

1.1 Hardware & Software Setup

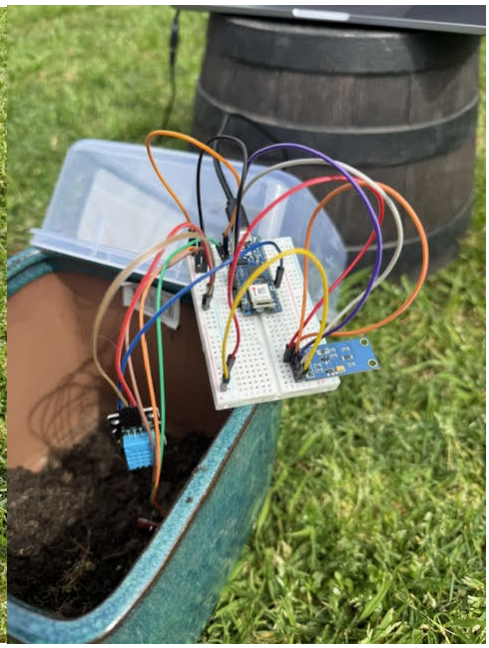
Sensors & Platform

- Arduino 33 IoT
- SparkFun soil moisture sensor (connected to A0)
- BH1750 ambient light sensor (I2C – SDA/A4, SCL/A5)
- DHT11 temperature/humidity sensor (digital, connected to D2) – added for the Assessment 3 revised study to enable VPD calculation
- Each sensor is connected to the ground rail on the breadboard which is wired to the Nano's 3.3V rail
- Communication – serial, through USB to laptop

Software & Storage

- An Arduino sketch is used to read all three sensors within a loop – printing readings to the serial monitor. It has a simple format: *Soil: [value] | Lux: [value] | Temp: [value]C | Humidity: [value]%*
- A Python script is used to listen to the port, reads incoming data, applies timestamps and lastly appends the data to a CSV file that is session-specific
- Storage – local CSV





1.2 Data Collection Execution

Session structure

As indicated in 3P's revised design, there were four testing sessions across two conditions: two tested in direct sunlight and two in a dark room. Additionally, as stated, neutral session labels were used throughout the study to avoid any descriptive labels that may lead to revealing location, routine or privacy.

Moreover, the pre-watering baseline was used as a reference for the threshold point for each session individually – as each session would have different baseline levels. Then, once moisture levels drop to around 15% of the baseline point, the alert for a watering event being needed would be sent. So, in plain terms, alert sent when the 15% band is hit (calculated per session, off each session's dry baseline).

Collection log

Session	*Condition*	*Start*	*Watering Event*	*Peak*	*End*	*Duration*
A	Direct sunlight	11:54:04	11:55:56	~795 (11:57:02)	15:00:40	~3h 6m
B	In the dark	23:03:32	23:03:40	~796 (23:10)	08:18:20 (next day)	~9h 15m
C	Direct sunlight	10:15:02	10:15:18	~642 (10:20)	13:10:00	~2h 55m
D	In the dark	23:00:02	23:00:12	~851 (23:10)	07:45:00 (next day)	~8h 45m

Outcome summary

From the data and results obtained from the testing, it can be observed that in session A, the moisture reading declined from its peak of ~795 (via watering event) back down to near baseline levels of around ~330 (close to the dry baseline levels of ~326) – indicating that a near-complete decline cycle was successfully captured.

The moisture decline in moisture reading observed in sessions B and D (dark room conditions) were marginal, dropping from their peaks of ~796 and ~851 down to just ~781 and ~833 respectively, across their 8+ hour testing windows.

Session C demonstrated moisture decline from the peak of ~642 down to the starting baseline level of ~373 by the end of the session and reached the 15% above-baseline threshold at around 13:00:00 – approximately 2h 45m after the watering event.

While singular readings in session A and C may have touched the 15% above-baseline threshold, the 10-minute block-averaged data that was implemented was designed to

remove any individual noisy readings and instead report sustained change in moisture levels.

Deviations from Assessment 3 protocol, with justification

- Since I didn't have four identical plants, I instead opted to use plant soil in a potted plant, just without the plant itself to measure moisture rate decline
- I had to reduce watering event volume (which was also held constant across each test) as my previous watering amount caused the decline to plateau without returning to near-baseline levels in a timeframe that would be practical enough for this study

Data quality report

Across each of the four testing sessions:

- there were no rows that were removed from the raw datasets
- there were no malformed rows found
- sensor-error readings (DHT11 temperature reaching implausible values like 2300°C for example) were removed from sessions B, C and D (2 in each), none in session A

Final row counts for each session (raw and processed):

	Raw	*Processed (aggregate 10-minute bocks)*
A	5,525	20
B	16,406	56
C	5,181	19
D	15,542	53

Session-specific notes

Session A:

- There is a sun-position shift evident around the 12:13-13:01 mark. This is flagged in the dataset as a condition change instead of being excluded – as a result, lux dropped down to ~5,745 during this window before being repositioned again back into direct sunlight.

Session B and D:

- No condition-change events to be noted, conditions remained the same in both testing sessions overnight. Two DHT11 sensor-error readings were removed from each testing session.

Session C:

- The watering event for this session was poured the same amount as each other session, however, there was no sharp rise as water trickled down near the sensor – causing a slower build up. I felt that while I could redo the test with new soil, that results produced would be the same either way as moisture decline was what was being captured anyway – as the moisture-decline phase is irrespective of how the initial watering-event rise occurs.
- Additionally, during this session, the sun was not as harsh as session A's conditions, lux levels were generally lower as well compared. Both sessions A and C were under direct sunlight, just varying intensity. Two DHT11 sensor-error readings were excluded here as well.

1.3 Data Processing Pipeline

Load

The raw CSV files obtained from each testing session were loaded into a pandas DataFrame using pd.read_csv(), which kept each session's data separate before being combined later on.

Validate

The DataFrames individually were checked for any missing values, duplicate timestamps and data type consistency – making sure that sensor columns were numeric rather than strings.

There were no malformed rows found in either of the four testing sessions.

Clean

There were no rows that were removed from the raw dataset themselves, and the full raw record is maintained for each session. However, a single flag column was added to mark data points that were in relation to environmental conditions changing – Session A's sun shift. I chose to keep this data and include it in the decline-rate calculation, and instead flag those data portions and discuss during the interpretation sections.

Transform

There were two columns added to each dataset in the sessions, deriving the following:

- VPD (vapour pressure deficit) – calculated from the temperature and humidity sensor data, included in the 3P revised design
- Decline rate – using the change in soil moisture values between consecutive aggregate time blocks for calculation

Aggregate

As discussed in the 3P feedback sections, to address the autocorrelation issues that were identified, each session's readings were instead aggregated into fixed 10-minute time blocks. The mean was calculated per block for each of the soil moisture, lux, temperature/humidity and VPD – reducing the testing sessions from multiple hours down to smaller data portions that are better suited for regression analysis.

Threshold Application

As mentioned previously, each session's processed dataset was then checked with its own specific threshold band (baseline + 15%) instead of the exact starting baseline value. This is designed to give the user enough time for them to act and water the plant, instead of giving them an alert when baseline levels are already reached.

Sessions A and C (direct sunlight) entered their 15% baseline threshold bands around 1h 54m and 2h 45m respectively after the watering events.
Sessions B and D (dark room) did not reach anywhere close to their 15% baseline threshold bands, even after running for 8+ hours – insinuating that the moisture decline rates in dark environments are far too slow to make threshold-crossing predictions practically within the timeframe of this study.

Export

The final processed CSV file includes data that has been cleaned, flagged and aggregated across each testing session.

To meet the 2 CSV file requirements, I combined the individual processed CSV files into a single combined_processed.csv file, and the four raw CSV files into a separate combined_raw.csv file for submission.

Part 2: Statistical Analysis & Visualisation

2.1 Descriptive Statistics

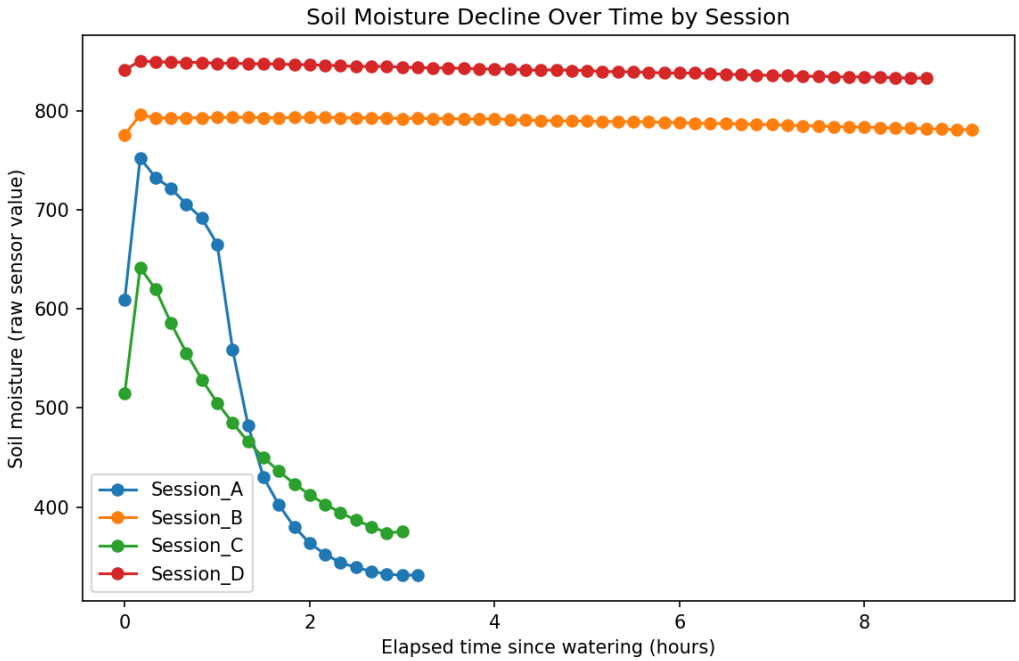
SESSION	*Condition*	*Soil (mean)*	*Soil (median)*	*Soil (std)*	*Lux (mean)*	*VPD (mean, kPa)*
A	Sun	493.1	416.1	165.6	16,413	2.26
B	Dark	789.3	790.2	4.4	0.27	1.09
C	Sun	470.4	449.9	85.1	5,454	2.02
D	Dark	841.8	841.9	5.1	0.33	1.79

Decline rate by conditions: Sessions A and C in the sun reported an average of 87.8 units/hr and 46.6 units/hr respectively. Dark sessions B and D averaged 0.6 units/hr and -0.9 units/hr respectively – basically indicating flat decline rates.

Correlation (block-level and within sun sessions): lux vs. decline rate: r = 0.049; VPD vs. decline rate: r = 0.104. Both showing negligible and weak correlation.

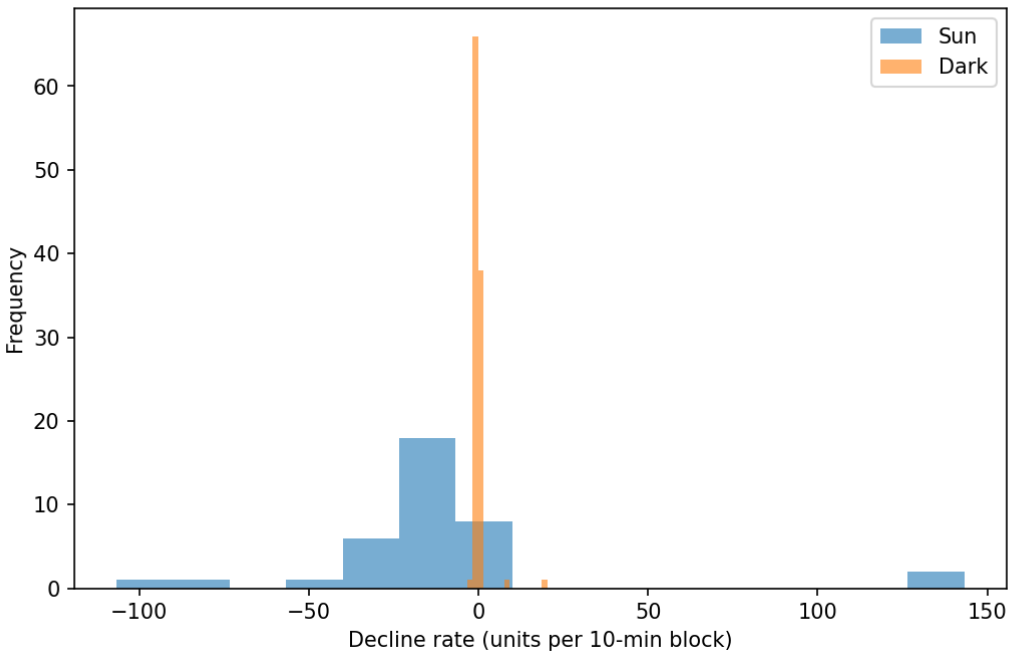
2.2 Visualisations

Plot 1 – Time Series



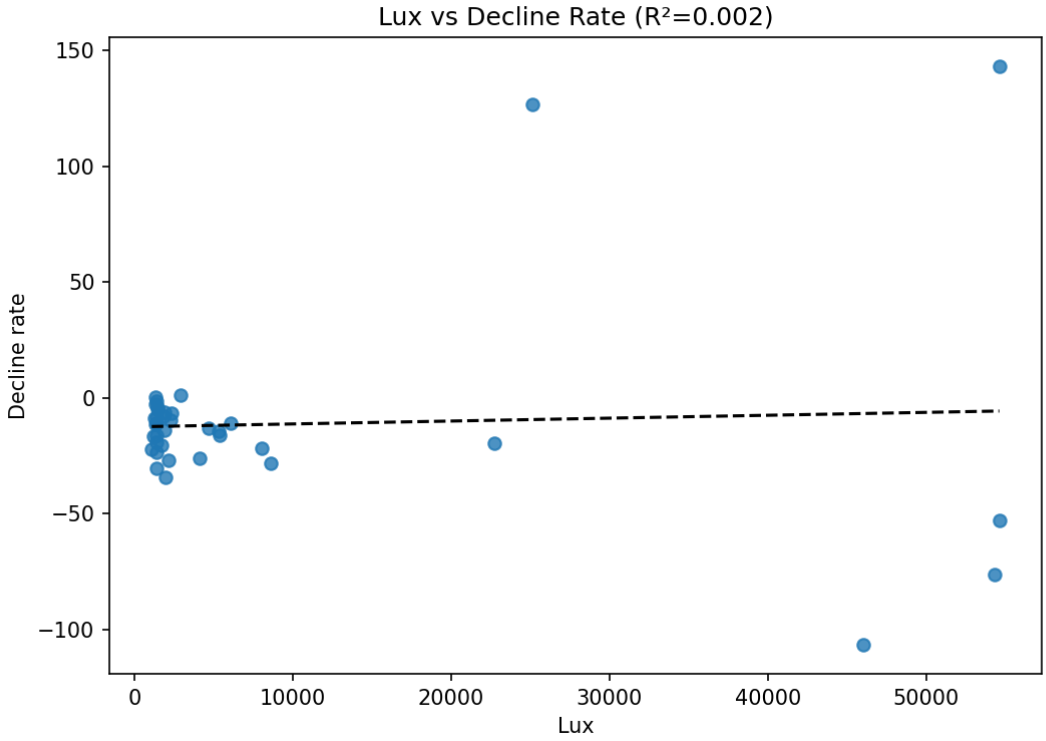
As seen above, there is a clear contrast between the sun and dark sessions demonstrating that both sun sessions decline quite sharply within around 3 hours, compared to that of the dark sessions which plateau essentially for the entirety of their 8+ hour sessions.

Plot 2 – Distribution



The distribution graph reinforces the H1 as the dark sessions are clustered around the zero mark, whereas the decline rate for the sun sessions are spread more widely – including positive during the watering-event rise in moisture levels.

Plot 3 – Relationship (lux vs. decline rate, sun sessions only)



The relationship graph shows a flat regression line where $R^2 = 0.002$, insinuating that there is basically no relationship between lux levels and decline rate within the sun sessions.

2.3 Statistical Testing & Hypothesis Validation

H1 — Sunlight accelerates decline rate

Analysing the data received, there is sharp contrast between the decline rates in the sun sessions and the dark sessions. Sun sessions declined at 87.8 units/hr in session A and 46.6 units/hr in session C. Dark sessions were essentially flat with 0.6 units/hr in session B and -0.9 units/hr in session D.

By looking at the 10-minute block decline rates, an independent t-test was carried out giving $t = -1.67$ and $p = 0.104$, which if we look at the usual 0.05 level, is not significant statistically even though there was a large gap between conditions. This observation is consistent with what the GEM had flagged in the 3P report, where because only 2 independent tests per condition were conducted, block-level testing would be weaker in providing proper statistical meaning.

As a whole, H1 proves to be well supported by the results obtained, however, the sample size of $n = 2$ per condition means that it isn't statistically significant.

H2 — Lux as a leading indicator

Observing the sun sessions in particular, the lux factor showed insignificant correlation with the block-level decline rate – $r = 0.049$, $R^2 = 0.002$. As a result, this doesn't support H2 as I originally thought and a possible explanation of this may be that the decline rate is driven by remaining moisture more than the current light-level being detected. This is evident in session A where the sensor was at its maximum limit during the early period where moisture decline was fastest – basically removing any variance that could have revealed a relationship between the two. Therefore, H2 is not supported by the results received in the tests at the within-session block level.

H3 — Threshold-crossing prediction within ±20%

Of the four testing sessions, only two captured the threshold crossing – sun sessions A and C.
Session A with a mean lux of 16,362 crossed the threshold at around 1h 54m and session C with a mean lux of 4,659 crossed the threshold at around 2h 45m. While this may seem consistent with the H2 and H3 theories where higher light intensity could result in a faster time-to-threshold result, the fact that there are only two genuine threshold crossing events being recorded, there is not enough evidence to validate a predictive model with significant statistical meaning.
As a result, H3 cannot be validated formally, instead it can be observed as being qualitative – something that the GEM also recognised as feedback in task 3P on sample size.

2.4 Data Interpretation Report

This study examined a few things: H1 – whether direct sunlight accelerates soil moisture decline, H2 – if ambient light levels can be used as a leading indicator of that decline, and H3 – whether we can produce a threshold-crossing prediction that is accurate within ±20% by using the light data captured across the four testing sessions.

Analysis of the results demonstrated strong support for H1 compared to the other hypotheses studied. It was seen that sun sessions declined at 87.8 and 46.6 unites/hour, compared to that of flat rates seen in dark sessions being 0.6 and -0.9 units/hour. Additionally, a block-level t-test also did not reach significance at p=0.104 – reflecting the fact that there were only two sessions run per condition. This aspect is a limitation of the study that was acknowledged and flagged in the 3P report in the GEM feedback portion.

H2 as a whole was not supported with the data collected during the testing sessions. Analysis showed that lux levels did not have any correlation with block-level decline with the rate being R^2=0.002. This insinuates that light levels cannot be used to predict the moisture loss as it is happening in any meaningful way – theorised due to decline rates following their own decay pattern that may be driven by remaining moisture and the fact that lux values were at their highest reading capacities, thus removing variance for relationship detection.

Since two sessions out of the four produced actual threshold-crossing events, H3 could only be assessed through the datasets produced in these sessions alone. The two dark sessions, while having much longer 8-9+ hour testing windows, did not approach the threshold whatsoever – concluding that moisture level decline in rooms with no sunlight is much too slow for a practical prediction given the timescale of which to complete the study in. However, looking at the two sun sessions, we can determine that the higher average lux did correspond to a faster threshold crossing time – giving us a proper result, but still one that cannot be seen as a validated outcome given the two testing sessions conducted in the sun.

In terms of limitations, again, the sample size of two sessions per condition was clearly one that arose multiple times during analysis. Alongside this, the DHT11 also demonstrated lower measurement accuracy than that of the DHT22 which was brought up in the GEM's feedback. The amount of water used in the watering events were reduced to ensure that threshold-crossing would actually be practical and feasible with the assessment timeframe. The study also utilised bare plant soil in place of four live plants, as well as session A having the sun-position shift mid testing window.

Regardless of the constraints encountered, I believe that the core findings were practical and still stand. It can be concluded that light condition does have a considerable effect on the decline of moisture speed which also supports the idea that

monitoring light levels actually adds value to detection, rather than just using moisture-only detection.
Refinements that can be implemented in the future can be to employ higher resolution temperature/humidity sensors, implement simultaneous testing rigs and furthermore, add more testing sessions to improve statistical validity with a longer study period.

Acknowledgments

No generative AI was used in this report.

Week 8 Activity

Student name: Taj Chehhal

Student ID: 221475358

SIT225: Data Capture Technologies

Activity 8.1: Using smartphone to capture sensor data

The **Arduino IoT Remote** phone application lets you control and monitor all of your dashboards in the Arduino Cloud. With the app, you can also access your phone's internal sensors such as GPS data, light sensor, IMU and more (depending on what phone you have).

The phone's sensor data is automatically stored in Cloud variables, which you can also synchronize with other Things such as custom thing in Python board. This means your phone can become a part of your IoT system, acting as another node in your network.

In this activity, you will enable your smartphone to work as a custom device (like an Arduino board) and connect to your smartphone sensors such as accelerometers and GPS and streaming data to Arduino IoT Cloud dashboard.

Hardware Required

Your smartphone – compatible Android or iPhone

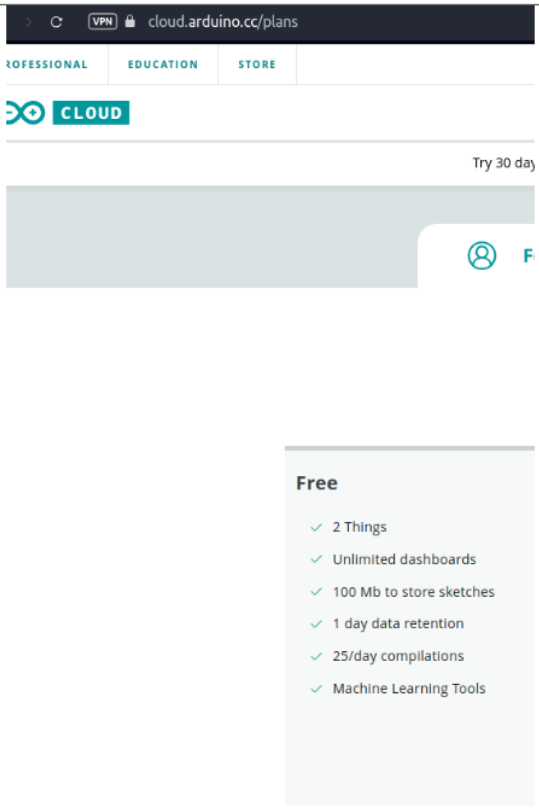
NOTE: *The IoT Remote app requires iOS 12.4 or later for iOS the version. If you are using Android, version 8.0 or later is required. Make sure the iOS or Android version on your device is up to date before downloading the app.*

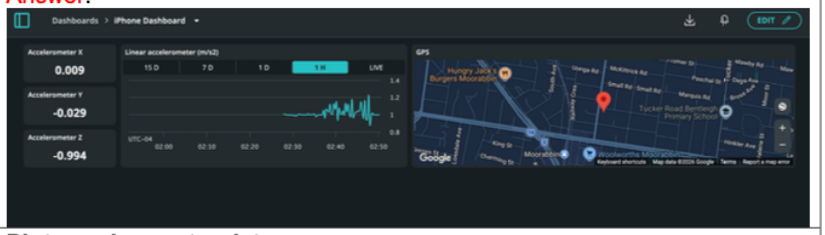
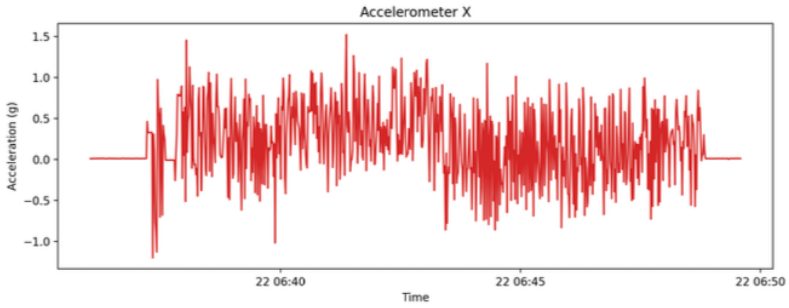
Software Required

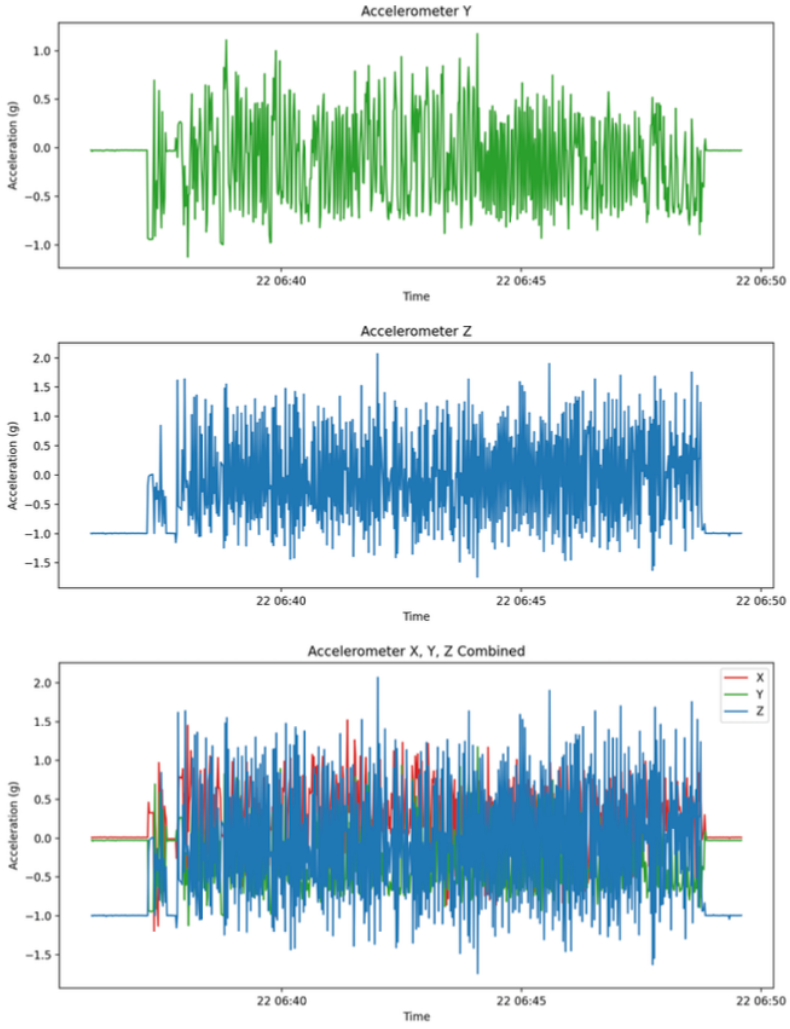
Android / iOS smart phone.
Arduino account
Arduino IoT Remote App (App Store or Google Play)
Python 3 (for custom Python Thing)

Steps

Step	Action
1	**Install App**: To use the Arduino IoT Remote app, visit Google Play / App Store and search for "Arduino IoT Remote". After installing the app, you will need to log in to your Arduino account. After you login, you will discover all your dashboards (if you have any), in the main menu. Based on the app version, home screen may vary. There will be 3 tabs at the bottom – Dashboards, Devices and Activity. You can follow the tutorial (https://docs.arduino.cc/arduino-cloud/iot-remote-app/getting-started).
2	**Add device**: Tap into the Devices tab. You will be able to create a new device. Alternatively, you can your profile (top right corner), in the settings section, you will see “Phone as device” which you can turn ON if it is OFF. There, you can select sensors in your smartphone such as accelerometer linear, accelerometer x/y/z and GPS among others. Note: A free account is enough for this experiment. If you are asked to upgrade your account, you can remove all other Things from your Arduino IoT Cloud account since the Free account allows at most 2 Things to configure, see below image.

	 The add device wizard will allow you to setup your sensor and also create a dashboard which you can see in your smartphone app. If you login to Arduino IoT Cloud in web browser, you can see the dashboard for your smartphone is already created.
3	**Keep your smartphone screen ON for a while**: Keep data coming through your smartphone for 10-15 minutes. During this time, keep moving your smartphone in a pattern so the accelerometer data can be analysed to discover the pattern. You can download data from the Ardiuno Cloud dashboard page by clicking on a download icon at top right corner which shows – Download historic data. A data download link will be sent to your account email from there you can download data.

	Question: Take a screenshot of your Ardiuno Cloud Dashboard where smartphone data is streaming and paste it here. Answer: 
4	**Plot accelerometer data**: The zipped data file you downloaded from the cloud contains separate files per variable including accelerometer_linear, accelerometer_x, accelerometer_y, accelerometer_z and Gps. Each file has 2 columns – time and value. Question: Open Jupyter Notebook by using command line, go to the data folder and write command ($ jupyter lab). Using Pandas, read CSV file and fetch the data column for accelerometer_x and plot it using Python plotting library (matplotlib or any other convenient for you). Repeat the plotting process for accelerometer y and z to have 3 separate graphs. Now create a fourth graph with all 3 variables x, y and z. Screenshot the 4 graphs and paste here. Answer: 

	
5	Question: Analyse accelerometer variables to find any repeating pattern. Remember that you were repeatedly moving your phone in a single pattern which should be manifested in the graphs. Justify your answer. Answer: All of the graphs show a repeating pattern – my hand doing a figure-eight movement for the data capturing window. As seen by the start and end of

	the graphs, there are moments of stationary data as the phone rests on the table. Then, there is movement, causing sharp rises in every axis. As the movement data is captured, it can be seen that the Z-axis shows the strongest signal and has faster oscillation – while the X and Y axis graphs show calmer oscillation. Thinking about how this outcome is from the figure-eight movement, I think this matches – as the wrist rotation would mean that the phones face would tilt toward and away from the body, leading to sensitive Z-axis data production, while the X and Y would send the side-to-side movement which in this case would be smaller in the loop.

Activity 8.2: Receive smartphone sensor data from Python script

You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that speaks Python. In activity 3.2, you have configured custom Python board and created a cloud variable that was synced to your Arduino Thing such as DHT22 sensor variables.

In this activity, you will need to synchronise smartphone's accelerometer x, y and z variables to Python script. If you can recall, you have already done a similar function in Activity 3.2.

Steps:

Step	Action
1	Configure Python board in Arduino Cloud and create a Thing where define 3 variables at a time and sync to corresponding accelerometer variable of smartphone Thing.
2	Write Python script to keep listening to data from the 3 variables to come through. You may need to create 3 call-back functions – a single function per variable (x, y and z).
3	Question: Keep storing each variable data in a separate file. Append each value with a timestamp so each data reading forms a comma separated line - <timestamp>, <data-value>. New data is written in a separate line. Keep storing them in a CSV file, where there will be 3 separate files. Screenshot your Python script here and screenshot the files opened side-by-side you have created and paste it here.

Answer:

Week 8 Activity > cloud_listener_separate_files.py

```python
import csv
from datetime import datetime
from arduino_iot_cloud import ArduinoCloudClient

DEVICE_ID = "4ba162f0-b760-4ab0-9bdf-1b71d06b5fce"
SECRET_KEY = "@TxbBNeTVfw1#Tm79z5OekQaB"

# One output file per axis
FILES = {
    "accel_x": "accel_x.csv",
    "accel_y": "accel_y.csv",
    "accel_z": "accel_z.csv",
}


def log_reading(variable_name, value):
    """Append <timestamp>,<value> as a new line to that variable's CSV file."""
    timestamp = datetime.now().isoformat()             # current time as a string
    with open(FILES[variable_name], "a", newline="") as f:  # "a" = append, keeps old rows
        writer = csv.writer(f)
        writer.writerow([timestamp, value])            # writes one "timestamp,value" line
    print(f"{variable_name}: {timestamp}, {value}")     # shows it in the terminal


# One callback per variable — runs automatically whenever that Cloud variable gets a new value from the synced phone data
def on_accel_x_changed(client, value):
    log_reading("accel_x", value)


def on_accel_y_changed(client, value):
    log_reading("accel_y", value)


def on_accel_z_changed(client, value):
    log_reading("accel_z", value)


# Set up the Cloud client and register the 3 variables
client = ArduinoCloudClient(
    device_id=DEVICE_ID,
    username=DEVICE_ID,
    password=SECRET_KEY,
)

# Registering a variable makes the client listen for it, and call the matching callback whenever a new reading comes in
client.register("accel_x", value=None, on_write=on_accel_x_changed)
client.register("accel_y", value=None, on_write=on_accel_y_changed)
client.register("accel_z", value=None, on_write=on_accel_z_changed)

print("Listening for accel_x/y/z updates... (Ctrl+C to stop)")
client.start()   # connects to the cloud and runs forever, firing callbacks as data arrives
```

accel_x

2026-09-22T17:47:31.769538	**0.8983612060546880**
2026-09-22T17:47:34.879865	-0.227783203125
2026-09-22T17:47:38.886548	0.4416656494140630
2026-09-22T17:47:49.225248	0.6831207275390630
2026-09-22T17:47:52.230626	-0.2914886474609380
2026-09-22T17:47:56.292950	-0.3368072509765630
2026-09-22T17:48:00.300427	0.0902252197265625
2026-09-22T17:48:04.358037	-0.38653564453125
2026-09-22T17:48:11.535353	0.1938323974609380
2026-09-22T17:48:27.164195	0.113067626953125
2026-09-22T17:48:40.671103	0.433258056640625
2026-09-22T17:48:44.676255	0.013275146484375
2026-09-22T17:48:48.758365	0.011199951171875

accel_y

2026-09-22T17:47:31.770623	**-0.3642578125**
2026-09-22T17:47:35.881796	-0.5446624755859380
2026-09-22T17:47:39.888664	0.4959716796875
2026-09-22T17:47:50.227495	-0.3341064453125
2026-09-22T17:47:53.232581	-0.479400634765625
2026-09-22T17:47:57.294755	0.1863861083984380
2026-09-22T17:48:01.302465	-0.8385772705078130
2026-09-22T17:48:05.360000	-0.015106201171875
2026-09-22T17:48:12.537298	-0.0386505126953125
2026-09-22T17:48:28.166821	-0.6736602783203130
2026-09-22T17:48:41.672583	-0.0411224365234375
2026-09-22T17:48:45.677990	-0.0280609130859375
2026-09-22T17:48:49.760290	-0.0263671875

	accel_z **2026-09-22T17:47:31.771080** **0.2015380859375** **2026-09-22T17:47:36.884224** 0.1335601806640630 **2026-09-22T17:47:40.890627** -0.74957275390625 **2026-09-22T17:47:51.229475** -0.5059051513671880 **2026-09-22T17:47:54.234610** 0.7218780517578130 **2026-09-22T17:47:58.296933** -0.69924926757812 5 **2026-09-22T17:48:02.304425** -0.1629791259765630 **2026-09-22T17:48:06.361885** 1.11279296875 **2026-09-22T17:48:13.539188** -0.774383544921875 **2026-09-22T17:48:29.167977** -0.72235107421875 **2026-09-22T17:48:42.674219** -1.39019775390625 **2026-09-22T17:48:46.679638** -0.999237060546875 **2026-09-22T17:48:50.765087** -0.9973602294921880
4	Question: Now manage 3 variable data so they can be stored in a single CSV file where each line consists of comma separated sensor values with a timestamp - <timestamp>, <x>, <y>, <z>. Store data once you gather 3 variables and repeat the process. Screenshot your Python script here and screenshot the file you have created opened and paste it here. Answer:

Week 8 Activity > cloud_listener_combined_file.py

```python
import csv
from datetime import datetime
from arduino_iot_cloud import ArduinoCloudClient

DEVICE_ID = "4ba162f0-b760-4ab0-9bdf-1b71d06b5fce"
SECRET_KEY = "@Txb0NeTVfw1#Tm79z5OekQa8"

OUTPUT_FILE = "accel_xyz.csv"   # single combined file

# Holds the most recent value received for each axis until we have all 3
# Starts empty and gets reset back to this after every row is written
latest = {"x": None, "y": None, "z": None}


def write_combined_row():
    """Write one <timestamp>,<x>,<y>,<z> line, then clear the buffer."""
    timestamp = datetime.now().isoformat()              # current time as a string
    with open(OUTPUT_FILE, "a", newline="") as f:        # "a" = append, keeps old rows
        writer = csv.writer(f)
        writer.writerow([timestamp, latest["x"], latest["y"], latest["z"]])  # one combined row
    print(f"{timestamp}, {latest['x']}, {latest['y']}, {latest['z']}")  # shows in terminal
    latest["x"] = latest["y"] = latest["z"] = None  # reset for the next set


# One callback per axis — each one fills in its slot in `latest`, and only writes a row once all 3 axes have a fresh value waiting
def on_accel_x_changed(client, value):
    latest["x"] = value
    if all(v is not None for v in latest.values()):     # true once x, y and z are all filled
        write_combined_row()


def on_accel_y_changed(client, value):
    latest["y"] = value
    if all(v is not None for v in latest.values()):
        write_combined_row()


def on_accel_z_changed(client, value):
    latest["z"] = value
    if all(v is not None for v in latest.values()):
        write_combined_row()


client = ArduinoCloudClient(
    device_id=DEVICE_ID,
    username=DEVICE_ID,
    password=SECRET_KEY,
)

# Registering a variable makes the client listen for it, and call the matching callback whenever a new reading comes in
client.register("accel_x", value=None, on_write=on_accel_x_changed)
client.register("accel_y", value=None, on_write=on_accel_y_changed)
client.register("accel_z", value=None, on_write=on_accel_z_changed)

print("Listening for accel_x/y/z and writing combined rows... (Ctrl+C to stop)")
client.start()   # connects to the cloud and runs forever, firing callbacks as data arrives
```

accel_xyz

2026-09-22T17:52:48.831253	**0.9074554443359380**	**-0.537445068359375**	**-0.0466766357421875**
2026-09-22T17:53:04.853072	-0.3103790283203130	0.2447662353515630	-0.93438720703125
2026-09-22T17:53:21.113939	0.5658721923828130	-0.588546752929688O	-0.6705322265625
2026-09-22T17:53:24.131074	0.7789459228515630	-0.5077667236328130	-0.3509979248046880
2026-09-22T17:53:41.865751	-0.26019287109375	-0.6971435546875	-0.393585205078125
2026-09-22T17:53:50.132519	-0.473785400390625	-0.873291015625	0.270050048828125
2026-09-22T17:53:54.139928	0.6185455322265630	-0.3381195068359380	-0.4860382080078130
2026-09-22T17:53:58.200229	0.131195068359375	-0.8441925048828130	0.8291473388671880
2026-09-22T17:54:02.257535	-0.4365081787109380	-1.0179290771484400	-0.0458984375
2026-09-22T17:54:06.264164	0.165313720703125	-0.2758636474609380	-1.035125732421880
2026-09-22T17:54:10.326056	1.0051727294921900	0.2371063232421880	-0.678253173828125
2026-09-22T17:54:15.384944	-0.873260498046875	-0.76495361328125	-3.0517578125e-05
2026-09-22T17:54:42.554979	0.5115203857421880	-0.7607269287109380	-0.080169677734375
2026-09-22T17:54:47.666642	0.197509765625	-0.860137939453125	-0.4011077880859380
2026-09-22T17:54:51.673298	0.4029083251953130	0.863433837890625	-0.497589111328125
2026-09-22T17:54:54.678706	-0.71258544921875	-0.5577850341796880	0.6280059814453130
2026-09-22T17:55:08.193873	0.0911712646484375	-0.0120391845703125	-1.1853179931640600
2026-09-22T17:55:12.200113	-0.2049102783203130	-0.277008056640625	-0.943572998046875

MakerSpace Soldering Induction + Soldering Process Report

Soldering Process Report

Introduction

Soldering is process commonly used in the electronic field to connect electrical and mechanical components together. Components such as leads, wires and PCBs are the most common connections in soldering, and the process is quite simple. Essentially, soldering involves using heat for the parts that are being joined together and within this, a filler metal called 'solder' is applied. This solder is basically melted metal and flows around the joint needed to be connected, before it then cools down and solidifies and securely forms a joint.
While the process is quite simple, for electronics especially, melting solder onto a component isn't always enough to form a reliable joint. Things such as surface cleanliness, heat application and solder flow all need to be considered and applied properly for a proper working connection to be formed. The following report outlines the materials and equipment required, as well as defining the soldering process itself and outlining common problems that can occur, in addition to any safety procedures that must be acknowledged.

Soldering Materials and Equipment

The main tool that is used in the soldering process is called a soldering iron or a soldering station. This iron features a metal tip that conducts heat and transfers it to the desired electrical connection. Additionally, there are many tip shapes and sizes that can be utilised depending on the specific component that is being soldered, and this means that certain tips can be more useful than others. The general rule being that the tip shouldn't be larger than the PCB or component but at the same time should provide enough surface area for the heat to be transferred properly and effectively.

Additionally, we also have the soldering wire. While it is common that modern electronics use lead-free solder alloys, lead-based solder can still be found. A flux core is contained within solder wire when using on electronics, and this flux essentially removes the oxidation from the metal surfaces, assisting the soldering process and improving solder flow and bonding with surfaces.

Alongside these two main tools, there are others that assist with the process:

- Soldering iron stand – a place for the use to place the solder to prevent any accidental burns and any nearby environment/surface from being melted
- Brass wool – removes any excess solder and maintains the temperature
- Fume extractor/ventilation – pulls the toxic fumes from the soldering process away from the user
- Solder sucker – a hand-held tool that removes any melted solder that is unwanted
- Side cutters – sometimes used to make sure component leads are trimmed
- Multimetre – checks for shorts, verifies connections and tests components

The Soldering Process

Before beginning with any soldering, we must ensure that the workspace and surrounding environment is clean and that the ventilation equipment is set up. The soldering equipment/soldering stand and if working with a through-hole PCB, have the component leads already inserted through in order for easy access from the board's underside and ready for soldering.

Once everything is clean and the workspace is clear, we can then turn the soldering iron on and wait for it to reach a good operating temperature – usually this is around 300-350 degrees Celsius for a lead-based solder, and 400-450 degrees Celsius for lead-free solders.

Before we begin any soldering, the tip should be cleaned and tinned. This essentially ensures that any residue from the previous soldering session is removed, as well as oxidation and flux residue, and tinning applies a bit of fresh solder to the tip that allows for better heat transfer from the iron to the joint.

Now we can begin the heating. The important part here is that the soldering tip should make contact with the PCB as well as the component lead at the same time – applying heat and joining the two components together. This is the essence of soldering – joining components by applying heat to them together.

We can then feed the solder wire into the heated joint once we have a good temperature with the connection. What happens here is that when the solder makes contact with the heated surfaces, it melts and is able to flow around the area. It is also important here to make sure that only enough solder is applied as needed and not excessive amounts, as this may flow over to other parts of PCB.

When we reach a point where we've applied enough solder, the wire is what should be removed first before the iron itself. Then allow the joint to cool on its own. Ensure that the component stays still and untouched during this drying process as we don't want any weak connections.

Next, inspect the connection and look for a proper connection between the component lead and the PCB pad – this can be confirmed when there is a visible controlled fillet around the connection. Check for any excess solder or gaps, as well as any accidental connections to surrounding PCB components.

If excess component leads are found, use the side cutters to trim them, then clean the soldering iron tip by using the brass wool and turn it off.

Common Problems

There are a few things to look out for when soldering.

If the component lead and PCB aren't heated sufficiently enough, then a cold solder joint can occur. This can also happen if there is movement during the drying process as the connection is cooling. The issue here is that this can lead to a poor electrical connection and may even cause circuit failure.

Worth mentioning again, that if too much solder is used, then solder bridging may occur – connecting unintentionally to other components pins/pads and may even cause short circuiting. On the other hand, if too little solder is used then a reliable connection may not be created between the PCB and component.

Additionally, if the heat applied is too high, then it could easily damage the PCB or electrical components, therefore, it is very important to know how much heat should be right for the electrical hardware that solder is being applied to.

If for any reason there is a mistake during the soldering phase, it can be removed by using a desoldering pump, then cleaned and re-soldered.

Soldering Safety

Due to the fact that very high temperatures are vital in the soldering process, the materials and soldering equipment must be handled very carefully, ensuring that the soldering iron is only held by its handle that is usually insulated, and when not in use, returned to its stand so that chances for accidents to occur is reduced.

It goes without saying that any items and food/drink should not be in the workspace, as well as making sure that the user washes their hands after a soldering session for any lead traces from the solder. Regarding toxic fumes, the fume extractor should be utilised to remove them, and safety glasses should also be worn in case any hot solder or component clippings make it near the eyes.

Moreover, if any electrical components are known to be sensitive to electrical charges, the user should use an anti-static mat or wrist strap so that the electrical components don't get damaged.

Conclusion

Soldering is a skill that is not only important to have, but also when done correctly, quite an easy and straight-forward task. It allows for the assembly and repairing of electrical circuits and can create new connections for components and PCBs. When attempting to create reliable solder joints, the preparation of the equipment, maintenance of equipment during the process, making sure that the heat applied is sufficient and applying the correct amount of solder are all crucial steps during the process.

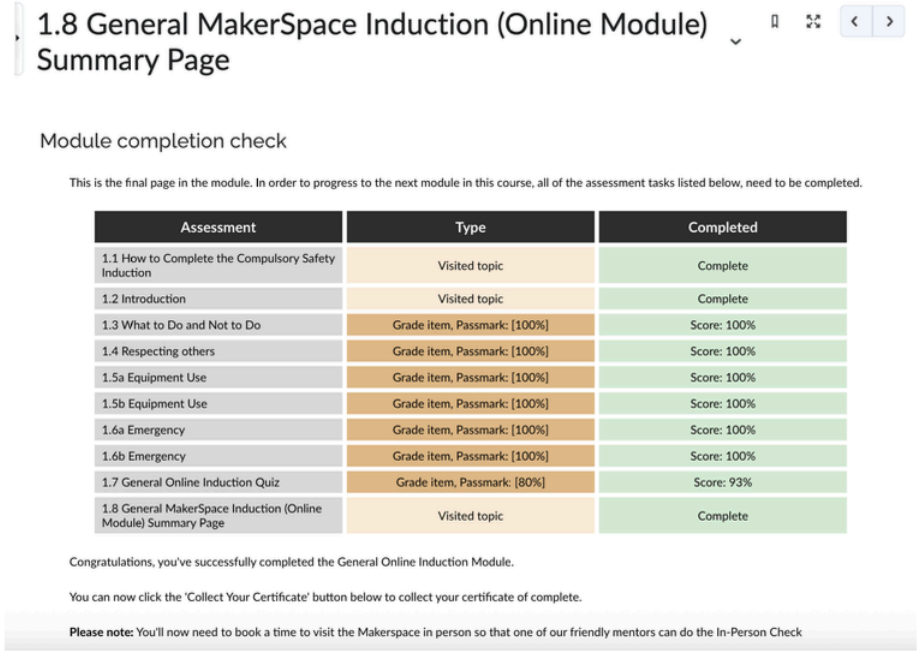
By understanding the common problems that one might run into, a user can identify those problems early and correct them before they turn into more complex issues later on. Additionally, by applying safe working practices, the user can ensure that the soldering process goes smoothly and no one is hurt.

To put it simply, a successful soldering process relies on heat control, solder flow and understanding the different phases and their timing, rather than going straight to soldering two components together.

References

- *Sild, S. (2022, August 31). Soldering Explained - Definition, Process, Types. Fractory.* https://fractory.com/soldering-explained/
- *Deakin University. (n.d.).* Soldering module *[Online course module]. SIT MakerSpace, CloudDeakin.* https://d2l.deakin.edu.au/d2l/le/content/1381894/Home
- *AIM Solder. (2025, August 11). What is Solder and How is it Used in Electronics Manufacturing? - AIM Solder. AIM Solder.* https://www.aimsolder.com/blog/what-is-solder-and-how-is-it-used-in-electronics-manufacturing/

General MakerSpace + Soldering Induction Completion Proof



1.8 General MakerSpace Induction (Online Module) Summary Page

Module completion check

This is the final page in the module. In order to progress to the next module in this course, all of the assessment tasks listed below, need to be completed.

Assessment	Type	Completed
1.1 How to Complete the Compulsory Safety Induction	Visited topic	Complete
1.2 Introduction	Visited topic	Complete
1.3 What to Do and Not to Do	Grade item, Passmark: [100%]	Score: 100%
1.4 Respecting others	Grade item, Passmark: [100%]	Score: 100%
1.5a Equipment Use	Grade item, Passmark: [100%]	Score: 100%
1.5b Equipment Use	Grade item, Passmark: [100%]	Score: 100%
1.6a Emergency	Grade item, Passmark: [100%]	Score: 100%
1.6b Emergency	Grade item, Passmark: [100%]	Score: 100%
1.7 General Online Induction Quiz	Grade item, Passmark: [80%]	Score: 93%
1.8 General MakerSpace Induction (Online Module) Summary Page	Visited topic	Complete

Congratulations, you've successfully completed the General Online Induction Module.

You can now click the 'Collect Your Certificate' button below to collect your certificate of complete.

Please note: You'll now need to book a time to visit the Makerspace in person so that one of our friendly mentors can do the In-Person Check

6.7 Soldering Module Summary Page

Module completion check

This is the final page in the module. In order to progress to the next module in this course, all of the assessment tasks listed below, need to be completed.

Assessment	Type	Completed
6.1 Safety Induction for Soldering	Visited topic	Complete
6.2 Soldering Introduction	Visited topic	Complete
6.3 Workflow: Theory Behind Soldering	Visited topic	Complete
6.4 Workflow: Soldering in Makerspace	Visited topic	Complete
6.5 Workflow: Things to Consider When Soldering	Visited topic	Complete
Soldering Induction Assessment Quiz	Grade item, Passmark: [80%]	Score: 92%

Congratulations, you've successfully completed the Soldering Online Quiz and Module.

You can now click the 'Collect Your Certificate' button below.

Please note: You'll now need to follow this link to book an in-person Soldering Safety Induction. Passing the soldering in person check is a requirement for being able to use the soldering stations unsupervised.